

PAMCA504-25MCA1020-
IPS10

April 10, 2026

ANN

Name: SHAMBHU PANDEY

Reg; No: 25MCA1020

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
[3]: diabetes=pd.read_csv("diabetes.csv",sep = ',')
diabetes
```

```
[3]:
```

		Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
	0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	
	..	...	...	...	...	...	...
763	10	101	76	48	180	32.9	
764	2	122	70	27	0	36.8	
765	5	121	72	23	112	26.2	
766	1	126	60	0	0	30.1	
767	1	93	70	31	0	30.4	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1
..	...	...	...
763	0.171	63	0
764	0.340	27	0
765	0.245	30	0
766	0.349	47	1
767	0.315	23	0

[768 rows x 9 columns]

[4]:

```
diabetes.isnull().sum()
```

[4] : Pregnancies 0

```

Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI 0 DiabetesPedigreeFunction 0
Age             0
Outcome 0 dtype: int64

```

[5] :

```
diabetes.describe()
```

[5]: Pregnancies Glucose BloodPressure SkinThickness Insulin \ count 768.000000
768.000000 768.000000 768.000000 768.000000 mean 3.845052 120.894531
69.105469 20.536458 79.799479 std 3.369578 31.972618 19.355807

```

15.952218 115.244002 min 0.000000 0.000000 0.000000 0.000000 0.000000
25% 1.000000 99.000000 62.000000 0.000000 0.000000 50% 3.000000
117.000000 72.000000 23.000000 30.500000 75% 6.000000 140.250000
80.000000 32.000000 127.250000 max 17.000000 199.000000 122.000000
99.000000 846.000000

```

```

          BMI DiabetesPedigreeFunction          Age      Outcome
count 768.000000          768.000000 768.000000 768.000000
mean  31.992578  0.471876  33.240885  0.348958 std  7.884160
0.331329 11.760232 0.476951 min  0.000000 0.078000 21.000000
0.000000 25% 27.300000 0.243750 24.000000 0.000000
50%  32.000000  0.372500  29.000000  0.000000 75%  36.600000
0.626250 41.000000 1.000000 max  67.100000 2.420000 81.000000
1.000000

```

[6]:

```
diabetes.head(15)
```

[6]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1
5	5	116	74	0	0	25.6
6	3	78	50	32	88	31.0
7	10	115	0	0	0	35.3
8	2	197	70	45	543	30.5
9	8	125	96	0	0	0.0
10	4	110	92	0	0	37.6
11	10	168	74	0	0	38.0

```

12          10  139  80  0  0 27.1
13          1   189  60  23 846 30.1
14          5   166  72  19 175 25.8

```

DiabetesPedigreeFunction Age Outcome

```

0          0.627  50  1
1          0.351  31  0
2          0.672  32  1
3          0.167  21  0
4          2.288  33  1
5          0.201  30  0
6          0.248  26  1
7          0.134  29  0
8          0.158  53  1
9          0.232  54  1
10         0.191  30  0
11         0.537  34  1
12         1.441  57  0
13         0.398  59  1
14         0.587  51  1

```

```
[8]: diabetes_raw = diabetes.drop(['Pregnancies','Outcome'], axis = 1)
```

```
diabetes_raw.replace(0,np.nan,inplace=True)
```

```
diabetes_raw.isnull().sum()
```

```
[10]:
```

```
[11]:
```

```
[11] : Glucose      5
```

```
BloodPressure 35 SkinThickness 227
Insulin          374
BMI    11 DiabetesPedigreeFunction    0
Age    0 dtype: int64
```

[12] :

```
diabetes_raw['Insulin'].unique()
```

```
[12]: array([ nan, 94., 168., 88., 543., 846., 175., 230., 83., 96., 235.,
        146., 115., 140., 110., 245., 54., 192., 207., 70., 240., 82., 36., 23.,
        300., 342., 304., 142., 128., 38., 100., 90., 270.,
        71., 125., 176., 48., 64., 228., 76., 220., 40., 152., 18.,
        135., 495., 37., 51., 99., 145., 225., 49., 50., 92., 325.,
        63., 284., 119., 204., 155., 485., 53., 114., 105., 285., 156.,
        78., 130., 55., 58., 160., 210., 318., 44., 190., 280., 87., 271., 129., 120.,
        478., 56., 32., 744., 370., 45., 194., 680.,
        402., 258., 375., 150., 67., 57., 116., 278., 122., 545., 75.,
        74., 182., 360., 215., 184., 42., 132., 148., 180., 205., 85., 231.,
        29., 68., 52., 255., 171., 73., 108., 43., 167., 249., 293., 66., 465.,
        89., 158., 84., 72., 59., 81., 196., 415., 275., 165., 579., 310., 61.,
        474., 170., 277., 60., 14., 95., 237., 191., 328., 250., 480., 265., 193.,
        79., 86., 326., 188., 106., 65., 166., 274., 77., 126., 330., 600., 185.,
        25., 41., 272., 321., 144., 15., 183., 91., 46., 440., 159., 540., 200.,
        335., 387., 22., 291., 392., 178., 127., 510., 16., 112.]])
```

[15]: `np.round(diabetes_raw['Insulin'].mean())`

[15]: `np.float64(156.0)`

```
[16]: diabetes_raw['Insulin'].replace(np.nan, np.round(diabetes_raw['Insulin'].
        s.mean()), inplace = True)
```

```
[19]: from sklearn.impute import SimpleImputer
        impute = SimpleImputer(strategy = 'median')
        diabetes_array = impute.fit_transform(diabetes_raw)
        diabetes_array
```

```
[19]: array([[148.    , 72.    , 35.    , ..., 33.6 , 0.627, 50.    ],
        [ 85.    , 66.    , 29.    , ..., 26.6 , 0.351, 31.    ]],
```

```
[183.    , 64.    , 29.    , ..., 23.3 ,    0.672, 32.    ],
...,
[121.    , 72.    , 23.    , ..., 26.2 ,    0.245, 30.    ],
[126.    , 60.    , 29.    , ..., 30.1 ,    0.349, 47.    ],
[ 93.    , 70.    , 31.    , ..., 30.4 ,    0.315, 23.    ]])
```

```
[22]: diabetes_df = pd.DataFrame(diabetes_array, columns = diabetes_raw.columns)
diabetes_df['Pregnancies'] = diabetes.Pregnancies diabetes_df['Outcome'] =
diabetes.Outcome diabetes_df
```

```
[22]:      Glucose BloodPressure SkinThickness Insulin      BMI \
0      148.0      72.0      35.0  156.0 33.6
1      85.0      66.0      29.0  156.0 26.6
2      183.0      64.0      29.0  156.0 23.3
3      89.0      66.0      23.0  94.0 28.1
4      137.0      40.0      35.0  168.0 43.1
..      ...      ...      ...      ...      ...
763     101.0      76.0      48.0  180.0 32.9
764     122.0      70.0      27.0  156.0 36.8
765     121.0      72.0      23.0  112.0 26.2
766     126.0      60.0      29.0  156.0 30.1
767      93.0      70.0      31.0  156.0 30.4
```

```
      DiabetesPedigreeFunction      Age Pregnancies Outcome
0              0.627 50.0      6      1
1              0.351 31.0      1      0
2              0.672 32.0      8      1
3              0.167 21.0      1      0
4              2.288 33.0      0      1
..              ...      ...      ...
763             0.171 63.0     10      0
```

764	0.340	27.0	2	0
765	0.245	30.0	5	0
766	0.349	47.0	1	1
767	0.315	23.0	1	0

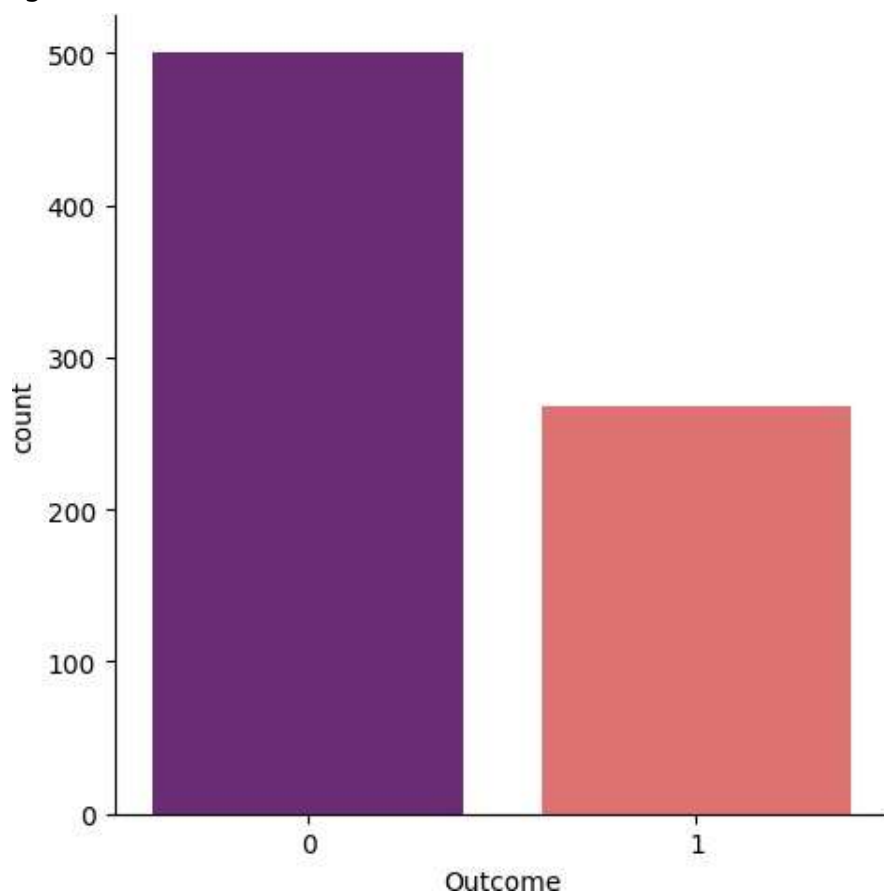
[768 rows x 9 columns]

```
[23]: sns.catplot(x = "Outcome", kind = "count", data = diabetes_df, palette='magma')
```

/tmp/ipykernel_423/143037878.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect. `sns.catplot(x = "Outcome", kind = "count", data = diabetes_df, palette='magma')`

```
[23]: <seaborn.axisgrid.FacetGrid at 0x7dba330e9f70>
```



[24]:

```
diabetes_df['Outcome'].value_counts()
```

[24] : Outcome

0 500

1 268

Name: count, dtype: int64

[25] :

```
df_0 = diabetes_df[diabetes_df['Outcome'] == 0] df_1  
= diabetes_df[diabetes_df['Outcome'] == 1]
```

[26] : df_1_upsample=resample(df_1, n_samples = 500, replace = True, random_state= 123)

[27] : diabetes_df1=pd.concat([df_0, df_1_upsample])

[28] : diabetes_df1['Outcome'].value_counts()

[28] : Outcome

0 500

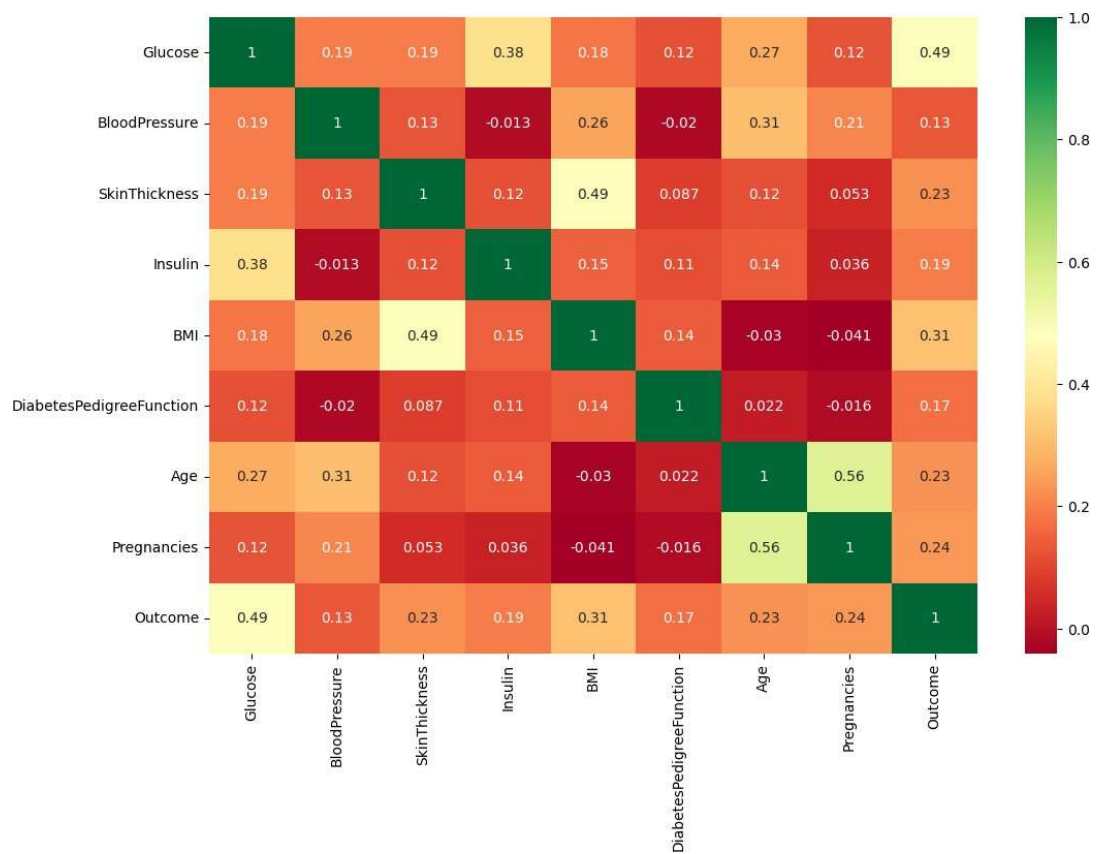
1 500

Name: count, dtype: int64

[29] : plt.figure(figsize=(12,8))

sns.heatmap(diabetes_df1.corr(), annot = True, cmap = 'RdYlGn')

[29] : <Axes: >



Splitting of Dataset

[31]

```
[30]: X=diabetes_df1.drop(['Outcome'], axis = 1)
      Y=diabetes_df1['Outcome']
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2,
                                                    random_state = 2)
```

:

```
[34]: classifier = MLPClassifier(hidden_layer_sizes=(5,25,25), alpha=0.01,
                                max_iter=200,activation = 'relu',solver='adam',random_state=1)
```

```
classifier.fit(X_train, Y_train)
```

[35]:

/usr/local/lib/python3.12/distpackages/sklearn/neural_network/_multilayer_perceptron.py:691:

ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet. warnings.warn(

```
[35]: MLPClassifier(alpha=0.01, hidden_layer_sizes=(25, 25, 25), random_state=1)
```

```
[36]: y_pred = classifier.predict(X_test)
```

```
[37]: def accuracy(confusion_matrix):
```

```
    diagonal_sum = confusion_matrix.trace()
```

```
    sum_of_all_elements = confusion_matrix.sum()    return
```

```
    diagonal_sum / sum_of_all_elements
```

```
[39]: from sklearn.metrics import confusion_matrix
```

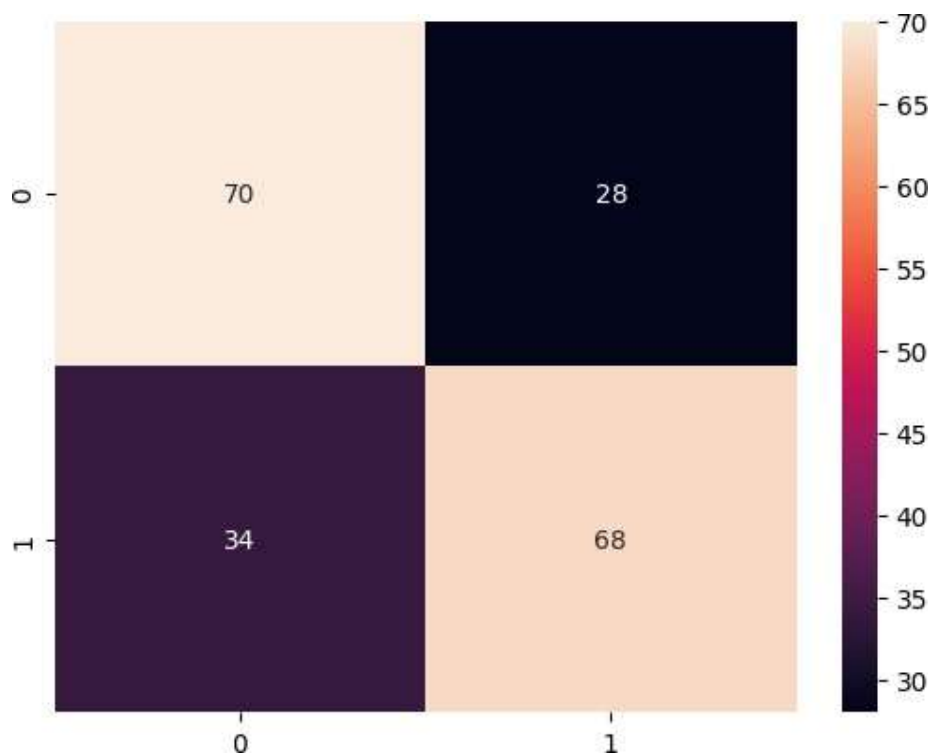
```
    cm = confusion_matrix(y_pred, Y_test)
```

```
    print('Accuracy of MLPClassifier: ', accuracy(cm))
```

Accuracy of MLPClassifier : 0.69

```
[40]: sns.heatmap(confusion_matrix(Y_test, y_pred), annot = True)
```

```
[40]: <Axes: >
```



```
[41]: from sklearn.metrics import classification_report
print(classification_report(Y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.67	0.71	0.69	98
1	0.71	0.67	0.69	102
accuracy			0.69	200
macro avg	0.69	0.69	0.69	200
weighted avg	0.69	0.69	0.69	200

```
[43]: classifier = MLPClassifier(hidden_layer_sizes=(100,100,50), alpha=0.
```

```
s1,max_iter=500,activation = 'relu',solver='adam',random_state=1)
```

```
[44]: classifier.fit(X_train, Y_train)
```

```
[44]: MLPClassifier(alpha=0.1, hidden_layer_sizes=(100, 100, 50), max_iter=500,
random_state=1)
```

```
[45]: y_pred = classifier.predict(X_test)
diagonal_sum = confusion_matrix.trace()
sum_of_all_elements = confusion_matrix.sum()
diagonal_sum / sum_of_all_elements
```

```
[46]: def accuracy(confusion_matrix):
```

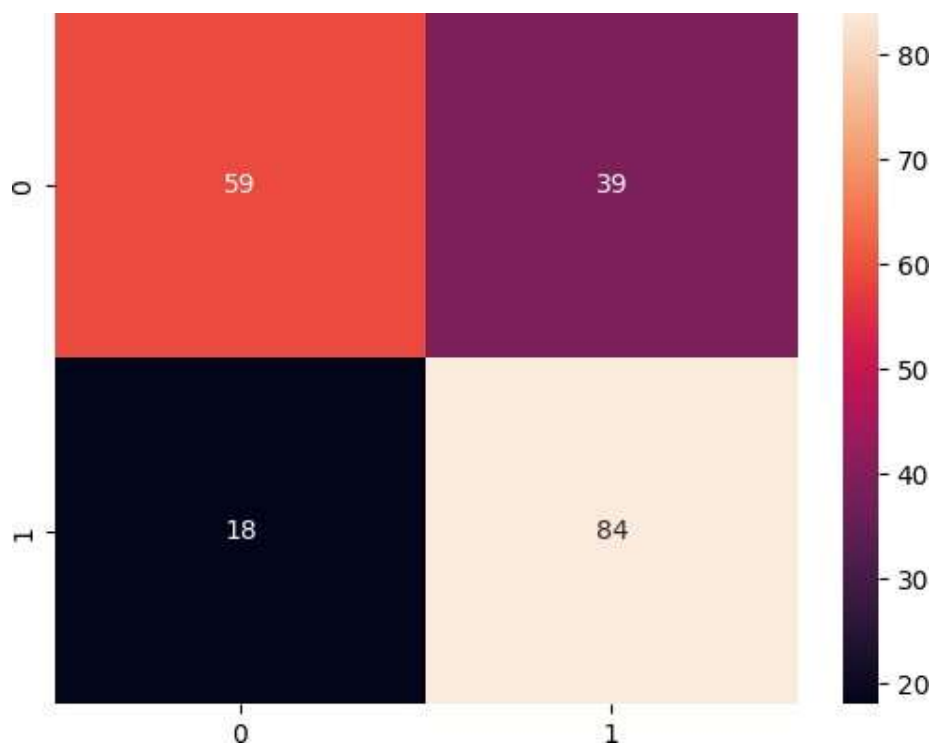
```
return
```

```
[47]: print("Accuracy of MLPClassifier : ", accuracy(cm))
cm = confusion_matrix(y_pred, Y_test)
```

Accuracy of MLPClassifier : 0.715

```
[48]: sns.heatmap(confusion_matrix(Y_test, y_pred), annot = True)
```

```
[48]: <Axes: >
```



[49] :

```
print(classification_report(Y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.77	0.60	0.67	98
1	0.68	0.82	0.75	102
accuracy			0.71	200
macro avg	0.72	0.71	0.71	200
weighted avg	0.72	0.71	0.71	200

[]:

